TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN - UTN FRM
BASES DE DATOS II - TRABAJO PRÁCTICO UNIDAD 7: NoSQL Y FOOD STORE

Integrantes: Nahuel Olmos, Facundo Cabrera, Ignacio Ariza y Agustin Pereyra
Repositorio GitHub: https://github.com/Nahuolmos/BDD-2PRO1-GRUPO-F.git

Modelo Documental y Justificación de Diseño

1. Descripción del Modelo Documental (pedidos)

Para representar el subconjunto de pedidos y detalle de pedidos de la aplicación *Food Store* en MongoDB, se definió la colección pedidos. La estructura se basa en el archivo datos_pedidos.json, desnormalizando las entidades relacionales de origen (pedido, detalle_pedido, cliente y producto) en un único documento BSON por transacción:


```
{
  "pedido_id": 1,
  "fecha": "2026-03-15T19:30:00Z",
  "estado": "ENTREGADO",
  "forma_pago": "EFECTIVO",
  "cliente": {
    "cliente_id": 1,
    "nombre": "Juan Pérez",
    "email": "juan.perez@email.com"
  },
  "detalles": [
    {
      "producto_id": 1,
      "producto_nombre": "Pizza Muzzarella",
      "cantidad": 2,
      "precio_unitario": 1000.00,
      "subtotal": 2000.00
    },
    {
      "producto_id": 3,
      "producto_nombre": "Coca Cola 1.5L",
      "cantidad": 2,
      "precio_unitario": 450.00,
      "subtotal": 900.00
    }
  ],
  "total": 2900.00
}
```


Componentes del Esquema:

- Campos Principales: pedido_id, fecha, estado, forma_pago y total.

- Objeto Embebido (cliente): Almacena datos del comprador (cliente_id, nombre, email).
- Arreglo de Objetos Embebidos (detalles): Contiene el desglose de productos comprados (producto_id, producto_nombre, cantidad, precio_unitario, subtotal).
- Identificadores Preservados: Se conservan explícitamente pedido_id, cliente_id y producto_id de PostgreSQL para mantener la trazabilidad entre ambos motores.

2. Reglas de Integridad Conservadas

1. Preservación de Precios e Importes Históricos: Los atributos precio_unitario y subtotal dentro del arreglo detalles registran valores históricos al momento exacto de la emisión del pedido. Si el precio del producto cambia en el catálogo general, el importe registrado en el pedido permanece inmutable.
2. Coherencia Transaccional del Total: El atributo total ($2.900,00 en el Pedido 1) debe ser exactamente igual a la suma aritmética de los subtotales del arreglo detalles ($2.000,00 + $900,00).

3. Justificación de Desnormalización

Se justificó embeber las líneas del detalle dentro del documento del pedido debido a que el patrón de acceso dominante en *Food Store* es la lectura del pedido completo (cabecera + renglones de compra). Al almacenar los detalles embebidos, se elimina la necesidad de realizar operaciones JOIN en tiempo de lectura, optimizando el rendimiento de la aplicación.

Evidencias de Migración y Consultas MongoDB

1. Migración de Datos

Se migraron exitosamente 5 pedidos completos a la colección pedidos en MongoDB utilizando el conjunto de datos de datos_pedidos.json mediante db.pedidos.insertMany().

2. Ejecución de Consultas en MongoDB

- Mostrar todos los pedidos (identificador y estado):

  ```
  db.pedidos.find({}, { pedido_id: 1, estado: 1, _id: 0 }).sort({ pedido_id: 1 });
  ```

Resultado de ejecución: [{ pedido_id: 1, estado: 'ENTREGADO' }, { pedido_id: 2, estado: 'ENTREGADO' }, { pedido_id: 3, estado: 'ENVIADO' }, { pedido_id: 4, estado: 'ENTREGADO' }, { pedido_id: 5, estado: 'PENDIENTE' }]

- Filtrar pedidos por estado (ENTREGADO):

  ```
  db.pedidos.find({ estado: "ENTREGADO" }, { pedido_id: 1, estado: 1, fecha: 1, total: 1, _id: 0 });
  ```

Resultado de ejecución: Recupera los pedidos con pedido_id: 1, 2 y 4.

- Buscar pedidos con producto_id = 1 (Pizza Muzzarella):

  db.pedidos.find({ "detalles.producto_id": 1 }, { pedido_id: 1, estado: 1, "detalles.$": 1, total: 1, _id: 0 });

Resultado de ejecución: Recupera los pedidos con pedido_id: 1, 3 y 4.

- Buscar pedidos con total superior a $5.000,00:

  db.pedidos.find({ total: { $gt: 5000 } }, { pedido_id: 1, estado: 1, total: 1, "cliente.nombre": 1, _id: 0 });

Resultado de ejecución: Recupera el pedido 4 (Total: $6.320,00, Cliente: Juan Pérez).
Justificación del Umbral: Se fijó un umbral de $5.000,00 para aislar las compras corporativas o de gran volumen que superan el ticket promedio del local ($1.500 - $3.000).

Consultas PostgreSQL

- Instrucciones ejecutadas en PostgreSQL (consultas_postgresql.sql):

Equivalente por Estado (ENTREGADO):

```
SELECT p.id AS pedido_id, p.estado, round(SUM(dp.cantidad * dp.precio_unitario)::numeric, 2) AS total

FROM pedido p

JOIN detalle_pedido dp ON dp.pedido_id = p.id

WHERE p.estado = 'ENTREGADO' AND p.id IN (1, 2, 3, 4, 5)

GROUP BY p.id, p.estado ORDER BY p.id;
```

Equivalente por Producto (producto_id = 1):

```
SELECT DISTINCT p.id AS pedido_id, p.estado

FROM pedido p

JOIN detalle_pedido dp ON dp.pedido_id = p.id

WHERE dp.producto_id = 1 AND p.id IN (1, 2, 3, 4, 5) ORDER BY p.id;
```

Tabla de Verificación Comparativa

Consulta	Cantidad en PostgreSQL	Cantidad en MongoDB	¿Coinciden los IDs?
Estado seleccionado (ENTREGADO)	3	3	Si
Producto seleccionado (producto_id = 1)	3	3	Si

Verificación de Coincidencia: Se obtuvo un 100% de coincidencia en los identificadores de pedidos recuperados en ambos motores (1, 2, 4 para el estado y 1, 3, 4 para el producto).

Diferencia Estructural de Respuesta: En PostgreSQL, la consulta por producto requiere joins entre pedido y detalle_pedido junto con DISTINCT para no duplicar filas. En MongoDB, el documento BSON devuelve la estructura completa con el arreglo embebido en una sola operación de lectura.

Contraste de Afirmaciones de IA

1. Afirmación 1: "MongoDB es intrínsecamente más rápido que PostgreSQL para consultar pedidos porque no utiliza operaciones JOIN."
    - Contraste Técnico: Aunque embeber datos en MongoDB elimina la necesidad de JOIN en lecturas de pedidos completos, la velocidad de PostgreSQL con claves primarias e índices B-Tree en detalle_pedido.pedido_id es del orden de los milisegundos. Asimismo, si en MongoDB no se crea un índice multiclave sobre detalles.producto_id, las búsquedas por producto requieren un escaneo completo de colección (COLLSCAN), siendo más lentas que en PostgreSQL.
2. Afirmación 2 de IA: "Embeber todos los datos del cliente en el documento del pedido es la mejor práctica NoSQL."
    - Contraste Técnico: Embeber la entidad completa del cliente duplica información variante (como e-mail o teléfono). Si el cliente actualiza sus datos, los pedidos anteriores quedan desactualizados o exigen actualizaciones masivas multidocumento. La buena práctica dicta desnormalizar únicamente datos inmutables (precios e importes históricos) y mantener la referencia cliente_id.

Conclusión del Proyecto

Para el subconjunto de pedidos de Food Store, se concluye que la mejor estrategia arquitectónica es implementar un enfoque de Persistencia Políglota (Polyglot Persistence), manteniendo PostgreSQL para el procesamiento transaccional y utilizando MongoDB como motor de lectura y consulta de historial.

MongoDB ofrece un rendimiento superior en la lectura de pedidos finalizados gracias al almacenamiento documental embebido (detalles y datos contables del pedido en un solo bloque BSON), eliminando la penalización de rendimiento de los JOIN relacionales en patrones de lectura masiva de alto tráfico.

No obstante, PostgreSQL resulta insustituible en la fase operativa de venta debido a dos criterios técnicos determinantes: la garantía de propiedades ACID con aislamiento estricto en operaciones de cobro y stock, y el control nativo de integridad referencial mediante claves foráneas. Migrar totalmente a MongoDB delegaría las validaciones de consistencia financiera a la capa de aplicación, elevando la complejidad del código. Por ende, se recomienda conservar PostgreSQL como base OLTP primaria y replicar los pedidos concluidos hacia MongoDB para lectura analítica e historial.